

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Отчёт №2

Сортировка

(по дисциплине «Алгоритмы и структуры данных»)

Выполнил:

Лабин Макар Андреевич,
АиСД 3.2, Р3231, 466449.

Проверил:

Смирнов Виктор Игоревич



г. Санкт-Петербург, Россия
2026

Содержание

1	Яндекс.Контест	1
1.1	Коровы в стойла	1
1.1.1	Описание решения	1
1.1.2	Асимптотическая сложность	1
1.1.3	Листинг кода	1
1.2	Число	2
1.2.1	Описание решения	2
1.2.2	Асимптотическая сложность	2
1.2.3	Листинг кода	2
1.3	Кошмар в замке	3
1.3.1	Описание решения	3
1.3.2	Асимптотическая сложность	3
1.3.3	Листинг кода	3
1.4	Магазин	4
1.4.1	Описание решения	4
1.4.2	Асимптотическая сложность	5
1.4.3	Листинг кода	5
2	Timus	6
2.1	Stone Pile	6
2.1.1	Описание решения	6
2.1.2	Асимптотическая сложность	6
2.1.3	Листинг кода	6
2.2	Hyperjump	7
2.2.1	Описание решения	7
2.2.2	Асимптотическая сложность	7
2.2.3	Листинг кода	7

1 Яндекс.Контест

1.1 Коровы в стойла

1.1.1 Описание решения

Применим стратегию бинарного поиска по ответу. Мы знаем, что ответ лежит в диапазоне от нуля до разности первого и последнего элементов исходного отсортированного массива. Предполагаем, что ответ лежит посередине данного отрезка. Если он подходит по условию, то двигаем левую границу к середине. Если нет — двигаем правую границу.

Проверить корректность предположений просто: достаточно пробежаться циклом по всему массиву и попытаться расставить коров хотя бы на предполагаемом расстоянии друг от друга. Если коров получилось не меньше, чем в условии — предположение верно, иначе — нет.

В ответ пойдёт максимальное из предположений искомого расстояния.

1.1.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(N \cdot \log N)$. Бинарный поиск происходит за логарифмическое число шагов, причём каждый шаг выполняется за линейное время из-за обхода всего массива.

1.1.3 Листинг кода

```
1 #include <iostream>
2 #include <vector>
3
4 int count_cows(std::vector<long>& arr, int n, long dist) {
5     int count = 1;
6     long last = arr[0];
7     for (int i = 1; i < n; i++) {
8         if (std::abs(last - arr[i]) >= dist) {
9             last = arr[i];
10            count++;
11        }
12    }
13    return count;
14 }
15
16 long bin_search(std::vector<long>& arr, int n, int k) {
17     long l = 0, r = arr[n - 1] - arr[0];
18     while (l + 1 < r) {
19         long mid = l + (r - l) / 2;
20         if (count_cows(arr, n, mid) < k) {
21             r = mid - 1;
22         } else {
23             l = mid;
24         }
25     }
26     if (count_cows(arr, n, r) >= k) {
```

```

27     return r;
28 }
29 return l;
30 }
31
32 int main() {
33     int N, K;
34     std::cin >> N >> K;
35     std::vector<long> arr;
36     for (int i = 0; i < N; i++) {
37         long item;
38         std::cin >> item;
39         arr.push_back(item);
40     }
41     std::cout << bin_search(arr, N, K);
42     return 0;
43 }

```

1.2 Число

1.2.1 Описание решения

Для решения задачи достаточно воспользоваться попарной лексикографической сортировкой строк в порядке убывания, а затем их конкатенировать. Такой подход гарантирует, что в старших разрядах окажутся цифры, имеющие больший лексикографический вес — а значит, и численный.

Стоит оговорить, что число всегда будет корректным с точки зрения записи. Это гарантирует условие: хотя бы в одной строке есть число с первой цифрой, отличной от нуля. Именно оно и будет первым за счёт большего лексикографического веса.

Для реализации попарной лексикографической сортировки строк воспользуемся стандартной сортировкой из библиотеки *STL* с изменённым компаратором, проверяющий лексикографический порядок конкретной пары строк.

1.2.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(N \cdot \log N)$. В алгоритме используется реализация сортировки из *STL*, которая имеет линейно-логарифмическую сложность согласно официальной документации.

1.2.3 Листинг кода

```

1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     std::vector<std::string> arr;
7     std::string line;
8     while (std::cin >> line) {

```

```

9     arr.push_back(line);
10 }
11 struct {
12     bool operator()(std::string a, std::string b) const {
13         return a + b > b + a;
14     }
15 } customLess;
16 std::sort(arr.begin(), arr.end(), customLess);
17 for (std::string& s : arr) {
18     std::cout << s;
19 }
20 std::cout << "\n";
21 return 0;
22 }

```

1.3 Кошмар в замке

1.3.1 Описание решения

По условию задачи, на стоимость строки влияет каждая пара уникальных одинаковых символов. Если они встречаются чаще, чем два раза — третий и далее символы не вносят свой вклад, их можно размещать как угодно.

Значит, при вводе исходной строки стоит проанализировать её и подсчитать статистику по каждому символу: сколько раз он встречается в строке. Если он появился хотя бы два раза, его можно использовать для повышения стоимости строки.

Далее в задаче говорится, что стоимость считается как сумма произведений веса пары символов на максимальное расстояние между ними. Поступим жадно: пару символов, которая весит больше, разместим на большее расстояние. Чтобы этого достичь, будем размещать их по принципу палиндрома: i -ую по весу пару символов разместим на i -ое и $N - i$ -ое места. Оставшиеся символы добавим в центр в произвольном порядке — на стоимость строки это не повлияет.

1.3.2 Асимптотическая сложность

Временная сложность решения — линейная $O(N)$. В алгоритме самые затратные по времени операция — посимвольная обработка строки: на каждый символ отводится константное число операций.

Полезная работа по максимизации стоимости строки выполняется за константное время, поскольку зависит от алфавита, который используется во входной строке — последний фиксирован по условию задачи.

1.3.3 Листинг кода

```

1 #include <algorithm>
2 #include <iostream>
3 #include <unordered_map>
4 #include <vector>

```

```

5
6 using letter = struct {
7     char c;
8     long v;
9 };
10
11 int main() {
12     std::string word;
13     std::cin >> word;
14     std::unordered_map<char, long> occur;
15     for (char c : word) {
16         occur[c] = occur.count(c) ? occur[c] + 1 : 1;
17     }
18     std::vector<letter> costs;
19     struct {
20         bool operator()(letter a, letter b) const {
21             return a.v > b.v;
22         }
23     } customLess;
24     for (int i = 0; i < 26; i++) {
25         letter l;
26         l.c = 'a' + i;
27         std::cin >> l.v;
28         costs.push_back(l);
29     }
30     std::sort(costs.begin(), costs.end(), customLess);
31     std::string pair, rest;
32     for (letter l : costs) {
33         if (occur.count(l.c) == 0) {
34             continue;
35         }
36         if (occur[l.c] > 1) {
37             pair += l.c;
38             for (int i = 0; i < occur[l.c] - 2; i++) {
39                 rest += l.c;
40             }
41         } else {
42             rest += l.c;
43         }
44     }
45     std::cout << pair << rest << std::string(pair.rbegin(), pair.
46         rend());
47     return 0;
48 }

```

1.4 Магазин

1.4.1 Описание решения

Воспользуемся жадной стратегией: максимизируем стоимость самых дешёвых позиций в каждом чеке. Для определённости стоит отсортировать данный

массив по возрастанию.

Если размер чека будет больше k , то массив получится разделить на меньшее число чеков без потери скидки. Ещё при таком подходе большой чек получит более дешёвые позиции, что уменьшает суммарную скидку. По этой причине не имеет смысла делить массив на части, кратные k .

В итоге остаётся делить массив на чеки размера k (*кроме, возможно, последнего*), начиная с правой части. В скидку пойдут крайние левые позиции из каждого чека размера k — она и будет максимальной.

1.4.2 Асимптотическая сложность

Временная сложность решения — линейно-логарифмическая $\mathcal{O}(N \cdot \log N)$. В алгоритме используется реализация сортировки из *STL*, которая имеет линейно-логарифмическую сложность согласно официальной документации.

1.4.3 Листинг кода

```
1 #include <algorithm>
2 #include <iostream>
3 #include <vector>
4
5 int main() {
6     int n, k;
7     std::cin >> n >> k;
8     std::vector<int> arr;
9     long sum = 0;
10    for (int i = 0; i < n; i++) {
11        int item;
12        std::cin >> item;
13        arr.push_back(item);
14        sum += item;
15    }
16    std::sort(arr.begin(), arr.end());
17    for (int i = n - k; i >= 0; i -= k) {
18        sum -= arr[i];
19    }
20    std::cout << sum;
21    return 0;
22 }
```

2 Timus

2.1 Stone Pile

2.1.1 Описание решения

В задаче используется ограничение $0 \leq N \leq 20$, что намекает на использование алгоритма высокой сложности. Следовательно, можно попробовать перебрать всевозможные варианты неупорядоченных распределений элементов между двумя группами — их не более $2^{20} \approx 10^6$.

Для оптимизации использования памяти можно воспользоваться битовыми операциями. В операционных системах чаще всего выделяется 32 бит для `int`, поэтому он подойдёт для ограничений данной задачи.

Таким образом, на хранение вариантов распределения будет выделено хотя бы $2^{20} \cdot 2^5 \cdot 2^{-23} = 4$ МиБ, что допустимо для ограничений данной задачи.

Далее для каждого сгенерированного варианта при помощи битовых сдвигов вычисляем сумму элементов каждой группы. Если i -ый бит справа равен единице, то элемент принадлежит первой группе, иначе — второй.

После мы вычисляем модуль разности сумм и обновляем глобальный минимум в случае необходимости.

2.1.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(N \cdot 2^N)$. После считывания массива генерируются всевозможные варианты неупорядоченного распределения элементов по двум группам. Затем, для каждого варианта вычисляется искомая разность сумм двух групп за линейное время и обновляется глобальный минимум.

2.1.3 Листинг кода

```
1 #include <iostream>
2 #include <vector>
3
4 std::vector<int> add_options(std::vector<int> options) {
5     std::vector<int> new_options;
6     for (int o : options) {
7         new_options.push_back(o << 1 | 1);
8         new_options.push_back(o << 1);
9     }
10    return new_options;
11 }
12
13 int main() {
14     int N;
15     std::cin >> N;
16     std::vector<int> arr;
17     for (int i = 0; i < N; i++) {
18         int item;
19         std::cin >> item;
20         arr.push_back(item);
21     }
```



```

22     std::vector<int> opts = {0, 1};
23     for (int i = 0; i < N - 1; i++) {
24         opts = add_options(opts);
25     }
26     long ans = 2000000;
27     for (int o : opts) {
28         long sum1 = 0, sum2 = 0, curr;
29         for (int i = 0; i < N; i++) {
30             if (o >> i & 1) {
31                 sum1 += arr[i];
32             } else {
33                 sum2 += arr[i];
34             }
35         }
36         curr = abs(sum2 - sum1);
37         if (curr < ans) {
38             ans = curr;
39         }
40     }
41     std::cout << ans;
42     return 0;
43 }

```

2.2 Hyperjump

2.2.1 Описание решения

Переформулировав условие задачи, для данного массива нужно найти отрезок с максимальной суммой его элементов.

Учитывая ограничение $0 \leq N \leq 6 \cdot 10^4$, можно размышлять в сторону алгоритмов квадратичной сложности. Это означает, что мы можем перебрать всевозможные начала и концы отрезков. Но этого пока недостаточно для нахождения суммы элементов на конкретном отрезке.

Для решения такой задачи необходимо предварительно кешировать префиксные суммы элементов массива за линейное время. Это позволит оптимизировать расчёт суммы элементов отрезка с линейного времени до константного.

Остаётся фиксировать текущий максимум суммы на каждой итерации, чтобы получить глобальный максимум.

2.2.2 Асимптотическая сложность

Временная сложность решения — квадратичная $\mathcal{O}(N^2)$. В начале работы алгоритма мы за линейное время создаём массив префиксных сумм и заполняем его. Затем для каждой неупорядоченной пары элементов массива выполняем константное количество операций.

2.2.3 Листинг кода

```

1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     int N;
6     std::cin >> N;
7     std::vector<int> arr;
8     std::vector<long long> prefix;
9     prefix.push_back(0);
10    for (int i = 0; i < N; i++) {
11        arr.push_back(0);
12        prefix.push_back(0);
13    }
14    for (int i = 0; i < N; i++) {
15        std::cin >> arr[i];
16        prefix[i + 1] = prefix[i] + arr[i];
17    }
18    long long ans = 0;
19    for (int i = 1; i <= N; i++) {
20        for (int j = i; j <= N; j++) {
21            long long curr = prefix[j] - prefix[i - 1];
22            if (curr > ans) {
23                ans = curr;
24            }
25        }
26    }
27    std::cout << ans;
28    return 0;
29 }

```